Consider a particular area in your city. Note the popular locations A, B, C . . . in that area. Assume these locations represent nodes of a graph. If there is a route between two locations, it is represented as connections between nodes. Find out the sequence in which you will visit these locations, starting from (say A) using (i) BFS and (ii) DFS. Represent a given graph using an adjacency matrix to perform DFS and an adjacency list to perform BFS.

Name : Ganesh Balkrushna Soanwane

Roll No: 43

```
from collections import deque
locations = ['A', 'B', 'C', 'D']
location_index = {name: idx for idx, name in enumerate(locations)}
adj_matrix = [
    [0, 1, 1, 0],
    [1, 0, 0, 1],
    [1, 0, 0, 1],
    [0, 1, 1, 0],
]
adj_list = {
    'A': ['B', 'C'],
    'B': ['A', 'D'],
    'C': ['A', 'D'],
    'D': ['B', 'C']
}

def dfs_matrix(start):
    visited = [False] * len(locations)
```

```
    result = []

    def dfs(node):
        visited[node] = True
        result.append(locations[node])
        for neighbor in range(len(adj_matrix)):
            if adj_matrix[node][neighbor] == 1 and not visited[neighbor]:
                dfs(neighbor)

    dfs(location_index[start])
    return result


def bfs_list(start):
    visited = set()
    queue = deque([start])
    result = []

    while queue:
        node = queue.popleft()
        if node not in visited:
            visited.add(node)
            result.append(node)
            for neighbor in adj_list[node]:
                if neighbor not in visited:
                    queue.append(neighbor)
```

```
    return result

def menu():
    print("----Menu----")
    print("1. DFS (using adjacency matrix)")
    print("2. BFS (using adjacency list)")
    print("3. Exit")

while True:
    menu()
    choice = input("Enter your choice: ")
    if choice == '1':
        start_location = 'A'
        dfs_result = dfs_matrix(start_location)
        print("DFS Traversal (using adjacency matrix):", dfs_result)
    elif choice == '2':
        start_location = 'A'
        bfs_result = bfs_list(start_location)
        print("BFS Traversal (using adjacency list):", bfs_result)
    elif choice == '3':
        print("Exiting...")
        break
    else:
        print("Invalid choice, please try again.")
```

'''OUTPUT

gescoe@gescoe-OptiPlex-3020:~/SE divB $ python3 bfs.py

----Menu----

1. DFS (using adjacency matrix)

2. BFS (using adjacency list)

3. Exit

Enter your choice: 1

DFS Traversal (using adjacency matrix): ['A', 'B', 'D', 'C']

----Menu----

1. DFS (using adjacency matrix)

2. BFS (using adjacency list)

3. Exit

Enter your choice: 2

BFS Traversal (using adjacency list): ['A', 'B', 'C', 'D']

----Menu----

1. DFS (using adjacency matrix)

2. BFS (using adjacency list)

3. Exit

Enter your choice: 3

Exiting...

'''